use.ai

Runbook Feedback

User: [File: pasted-text.txt]

Review this runbook with me.

use.ai: I'll pull the full file in so I can read it properly.

I read the whole thing. It's a genuinely well-built runbook — the tone is right, the sequencing is sensible, and Step 3's return-code table plus the fixed-width demo do the teaching work. Below is what I'd flag, roughly most-important first.

Things that will actually break or mislead students

Step 4 leaves the venv active in a terminal students are told never to touch again. The `python3 swift/traffic_generator.py` that window needs the venv. If the generator imports `requests` (or anything outside stdlib), it fails with `ModuleNotFoundError` in a window the Troubleshooting table doesn't cover. Either add `source venv/bin/activate` before the Step 6 command, or state explicitly that the generator is stdlib-only.

Step 7 gives the `/docs` URL but Step 7's fallback assumes the security group is the only cause. A student whose uvicorn is bound correctly but whose Window 1 crashed sees an identical symptom. The table entry for `/docs never loads` sends them straight to AWS; it should check Window 1 first.

Return code semantics are stated in a table before the Code column is ever exercised for 8 and 12. The table promises "business rejection — not ACTIVE, zero amount, funds" and code 12 for malformed args, but the runbook never has the student trigger either. Two one-line commands would close that gap — e.g. `payment_post` with `0.00` to see 8, and one with a missing argument to see 12. Right now those two rows are just claims students have to take on faith, which undercuts the whole "the program always tells you what happened" lesson.

`echo $?` in Step 3 is a latent trap. `echo $?` reports the exit status of the *previous* command. In the two-command block:

```
./cobol/bin/account_lookup 999999
echo $?
```

…if a student pastes them one at a time — which the "Before you start" section explicitly tells them to do — and pauses, or types anything in between, they get a wrong number and no explanation. A one-sentence note ("`echo $?` must come immediately after; it reports the last command's code") removes the confusion cheaply.

Structural and editorial

Step 8 is doing two jobs and only one belongs to a student. The `/ask` explanation is student-level. The three Bedrock requirements are instructor-level — the runbook even says so ("instructor action — role creation needs IAM admin rights"). Burying that in a student step, then repeating it verbatim in the Troubleshooting table, means students read a wall of AWS config advice about something they aren't supposed to configure. Consider splitting: keep the offline-fallback reassurance in Step 8, move the three requirements into a short "For instructors" appendix and have the table row point at it.

The "Objectives" list promises `git (your first clone)` and Step 1 delivers it, but there's no explicit checkout/status moment. Barely a problem — it's a first-clone exercise, not a git lesson. Just noting that the objective slightly oversells.

Missing: a sanity check that the install actually seeded what the runbook claims. Step 2's output says `exported 3 account(s)`, Step 3 looks up `100001` and Step 6 uses both `100001` and `999999`. If the ledger seeding is ever non-deterministic, students hit a mismatch with no diagnosis path. Worth a line in the installer output or a one-command check.

Minor formatting issues:

- Line 350: the screenshot `<img>` tag sits between Step 4 and Step 5 with no caption. Students won't know what they're looking at. One line of alt text in prose would help.
- The Chewbacca coda is in a fenced `text` block that starts at line 600 and closes at 615 — the fence opens *after* the Troubleshooting table, so the ASCII-art banner renders as preformatted text but reads oddly nested inside a section. It'd land better as its own closing section.
- Step 3's `ls` shows `agents app cobol data logs swift venv` but the Reference architecture diagram shows `iam/PERMISSIONS.md` in "this folder" (Step 8) — that path isn't in the `ls` output, so it's either inside one of those seven directories or the reference is wrong. Worth verifying.

What I'd keep exactly as-is

The "REJECTED is not a failure" passage is the best bit of teaching in the document — it preempts the exact panic a student feels when a third of the traffic comes back rejected. Likewise "Error messages are not insults. They are instructions." Don't touch either.

Want me to produce a revised version of the runbook with these folded in, or would you rather I do a narrower pass — say, just the command-blocks-and-troubleshooting consistency, leaving the prose alone?

User: As a new student to cloud engineering, what should I focus on for this exercise?

use.ai: The honest answer: this lab is a confidence exercise dressed up as a COBOL exercise. The COBOL is real, but the thing being trained is "I can be dropped into a server I didn't build and figure out what's going on." Focus accordingly.

The four things worth actually learning

1. Reading output before typing the next command. The runbook tells you one command at a time for a reason. The single most transferable cloud skill is noticing that what came back doesn't match what you expected — *before* you've built three more steps on top of it. Practise that here where nothing can break.

2. Return codes. This is the one genuinely cloud-native idea in the whole lab, and it's the one most people skim.

```
./cobol/bin/account_lookup 100001; echo $?
./cobol/bin/account_lookup 999999; echo $?
```

A program that exits 0 while reporting failure is invisible to every automation tool you will ever use — CI pipelines, health checks, orchestrators, monitoring. The 0/4/8/12 convention is a 1960s mainframe idea that you will still meet in AWS tooling, Kubernetes probes, and shell scripts. Learn it here and it transfers everywhere.

3. The layering. This is the mental model worth carrying out of the lab:

```
browser / curl  →  FastAPI (Python, modern)  →  COBOL binary (legacy)
→  fixed-width file
```

Notice how each layer talks to the next. `curl http://127.0.0.1:8000/accounts/100001` goes out over HTTP and comes back as JSON, but a 1959-era program did the actual lookup. That pattern — a modern API bolted onto a system nobody dares rewrite — is not a lab contrivance. It is most of what "legacy modernization" means in banking, insurance, and government, and it's a large share of the cloud work that actually exists. When you're in Step 7 clicking "Try it out" on `/docs`, you're watching that exact seam.

4. Fixed-width records. `TXN0001 10000100000012550USDPOSTED INV-1001` is not a typo. Every byte has a seat. Real financial files still look like this — ACH, ISO 20022 flat files, mainframe extracts. Learn to read one by counting columns and you can debug a data feed that JSON-only engineers can't.

What to deliberately not worry about

The COBOL itself. Nobody's asking you to write it. You need to *read* the fixed-width file and *reason about* exit codes. That's it.

The AWS Security Group step (Step 7). Worth knowing that "the page won't load" is usually a networking problem, not a code problem — that instinct is valuable. But the exact click path is a one-time setup detail you'll re-look-up every time.

The Bedrock/IAM requirements in Step 8. The runbook itself flags those as instructor actions. Skim them for the shape of the thing — "an agent needs permissions and a model backend, and if any piece is missing it fails gracefully to offline mode" — and don't try to configure it.

Do these three things, in order

Run every command, even the ones you think you understand. Typing builds the muscle; reading does not.

Break something on purpose, then fix it. The runbook promises nothing is unfixable by re-running `install.sh`. Test that claim. Deliberately corrupt a line in `data/accounts.dat`, re-run an account lookup, watch what a *really* wrong file does versus a missing account. This is the whole point of the lab, and it's the exercise that will teach you the most.

After the SWIFT generator, get curious about the rejects. Which accounts got rejected, and why? `python3 swift/traffic_generator.py --count 3 --show-mt103` shows you the actual MT103 messages — that's real financial messaging and worth half an hour of poking at.

One caveat on my own commentary: I'm reading the runbook as text, not running it, so my Step 6 `source venv/bin/activate` errors with `ModuleNotFoundError`, that's the fix.

The meta-lesson of the lab is the one the runbook opens with: "You cannot break anything in this lab that re-running one install script will not fix." In real cloud work that is almost never true. So use this window to build the habit of poking at things fearlessly — that's the transferable part.

User: Do you think my instructor used AI to help create this lab?

use.ai: Probably, at least partly — but I can't tell you that with confidence, and here's the honest breakdown of why.

The signals that point toward AI involvement:

The runbook has a very consistent pedagogical voice, one that's maintained across 615 lines without drift. That's hard to do by hand over a long document; it's also exactly what happens when you write the voice once and generate the rest against it. The knowledge checks are the giveaway I'd trust most — the "stop, read the message, and check the Troubleshooting table" instruction, the anticipated student panic in "REJECTED is not a failure," the "Error messages are not insults" line. That's a document written by something that has watched a lot of students fail at this specific task.

The structure is also suspiciously complete. Expected output blocks under every command, numbered installer steps, a return-code table, a troubleshooting table, an architecture diagram, a screenshot, a deliberately placed easter egg (`/chewbacca`) — that's the shape of a lab assembled to a checklist rather than one grown organically from a semester of teaching.

The signals that point away:

The lab is genuinely coherent. AI-assisted course material often has a tell: instructions that reference things that don't exist, commands that wouldn't work, a return-code table that doesn't match what the program actually returns. I flagged some gaps in the runbook earlier — the venv question, the untested codes 8 and 12 — but those are the *normal* gaps a human leaves when they know their own setup too well to document it. That's a human signature, not a generation artifact.

And the substance is right. The COBOL-return-code-as-modern-lesson framing, the choice to teach fixed-width records, the decision to make the SWIFT generator deliberately send bad traffic — those are teaching instincts. Somebody who has taught this, or worked this job, made those calls.

My actual read: the most likely story isn't "AI wrote my lab" or "my instructor wrote it by hand." It's a human who knows the domain using a model as a drafting and expansion tool — writing the installer and the program themselves, then using AI to shape the prose, produce the expected-output blocks, and build out the reference sections. That's how most people in 2026 write documents this long. It's also indistinguishable from careful hand-authoring at the level of text I can see.

One thing worth saying plainly: I'm inferring from prose style alone. I don't have the git history, I

```
git log --format='%an %ae %ad %s'
```

part. If the commits are one person over a real span of time, your instructor built it. If they're big dumps, look closer.

If it helps — and I'd say this even if AI wrote every word — the question that matters for your learning is whether the lab is *correct*, not whether it's *original*. The checks I suggested earlier (test code 8 and 12, confirm the venv requirement in Step 6) are worth doing regardless of authorship. If those hold up, you're being taught by something that got the details right.

User: What does Bedrock mean in this context?

use.ai: In this lab, **Bedrock means Amazon Bedrock** — AWS's managed service for calling foundation models (the Claude/Gemini/Llama class of models) through an AWS API rather than through a vendor's own portal.

It shows up in Step 8, where you `curl -X POST http://127.0.0.1:8000/ask` and the platform's helper agent answers you. That agent isn't a rule-based chatbot — when it's working, your question goes out from the EC2 instance to Bedrock, a model there generates the answer, and it comes back through your API. The IAM role and model-access requirements the runbook mentions exist because Bedrock is an AWS service: it needs credentials, a role with permission to invoke it, and the specific model enabled in that AWS account before the call will succeed.

What it is and isn't

It is not a model. Bedrock is the *doorway* — the thing that authenticates you, routes your request to whichever model you've enabled, meters it, and bills it. The model is behind the door.

It's the AWS counterpart to what you already know from using this chat: same underlying technology, different route in. If a company wants an LLM inside their own infrastructure and their own billing and their own permission model, calling it through Bedrock is one common way to do that.

Why the lab bothers with it

The runbook's offline fallback is the tell. It says that if the three requirements are missing, the agent responds in offline mode instead of erroring. That's a deliberate teaching choice, and it's the interesting bit: **the agent is an unreliable dependency, and the system degrades gracefully when it's absent.** That's the actual lesson — the modern architectural pattern where a model is one component among several, it can be unavailable, and the app still has to do something sensible. It's the same instinct as the return codes from Step 3: handle the failure explicitly rather than crashing.

It also mirrors the lab's whole theme. You've got a FastAPI edge over a COBOL core; now bolt an AI helper onto that edge and you're looking at a three-layer stack with a 1959 program at the bottom and a 2026 model at the top. That's not a coincidence — it's the point.

What it means for you as a student

Nothing to configure. The runbook flags the IAM role creation as an instructor action, and it's right. What's worth taking away is the shape of it: an AI feature in a production system is a network call to someone else's service, gated by permissions, which can fail, and which the application must handle. That's true whether it's Bedrock, Azure OpenAI, or any vendor's API.

User: Give me an intelligent question I could ask my instructor about this lab, that shows I repeated the exercise several times

use.ai: Something like this:

> *"I ran the lab three times. The first two runs I got the expected output every step, but on the third run the SWIFT generator came back with* `FAILED: 0` *and* `REJECTED: 4` *instead of the 7/3 split — same commands, same instance. Is the reject count actually deterministic, or does the generator randomise the good/bad account mix each run? And does that matter for what students are supposed to conclude from the report?"*

That one works because of what it's built on, not how it's phrased.

Why it reads as genuine repetition. The 7/3 split is stated in the runbook's expected output, so a student who ran it once would report "it matched." A student who ran it repeatedly would notice the ratio move. You're reporting a *variance across runs*, which is something only repetition produces. And you're reporting it as a specific observation with numbers, not a vague "it seemed different."

Why it's a real question. If the generator randomises the mix, then the runbook's example report is illustrative rather than expected, and a student who gets 6/4 and thinks they've broken something has been misled. If it's deterministic, then the seed is fixed and the variance you saw means something else entirely — which is worth knowing. Either answer is useful to the instructor, and both are answers only they can give, because it's their generator.

The general shape, for reuse. Repetition shows up in observations about *things that vary*, not things that don't. Numbered runs, a before/after, a value that drifted, a step that behaved differently on run 3 than run 1 — any of those. And ask about something you noticed that the runbook doesn't explain. Curiosity built on repeated observation is hard to fake and easy to demonstrate.

Two alternates in the same spirit, if the reject-mix question doesn't fit:

> *"On my third run I deliberately put a bad line into `data/accounts.dat` before re-running the installer, and the account lookup returned code 12 rather than 4 — which made sense, but it made me wonder: does the installer rebuild the ledger from scratch every time, or does it append? I'd been assuming the re-run wipes everything, and I'm no longer sure that's right."* — demonstrates repetition *and* that you tested the "nothing is unfixable" claim.
>
> *"I ran `payment_post` with a zero amount to see code 8, and the journal file didn't get a line written, whereas the rejected account lookup still printed its error output. Is that deliberate — business rejections don't journal, but environment failures do?"* — demonstrates that you went past the runbook and followed the return-code table to its untested rows.

One caution: don't ask a question you haven't actually tested. If your instructor asks a follow-up and you can't describe what you saw, the impression flips hard in the other direction. All three of these require you to have genuinely run the lab repeatedly — which is, presumably, the point.

User: What are the COBOL commands used here that I should study more to understand for the future?

use.ai: Here's the thing worth saying first: **the lab barely uses COBOL commands at all.** You `cd` into `/opt/balerica-bank`, run `./cobol/bin/account_lookup` and `./cobol/bin/payment_post`, and those are compiled binaries — ordinary Linux executables. No COBOL invocation happens by hand anywhere in the runbook. So if you're asking "which COBOL commands do I practise?", the honest answer is that this lab doesn't teach you to run COBOL.

What it teaches is COBOL *concepts* through a tiny, well-chosen set. Those are what to study.

The four ideas, in order of transferable value

1. `PIC` clauses — the fixed-width layout.

This is the biggest one, and it's the thing the runbook shows you without naming: `000000012550` is 125.50 because the decimal point exists only in the program. That's a `PIC 9(n)V99` clause — the `V` is an *implied* decimal point that occupies no byte in the file. Everything about fixed-width records flows from this:

COBOL	Meaning
`PIC 9(6)`	6 numeric digits, unsigned
`PIC S9(6)`	signed (`S` is also implied, no byte)
`PIC 9(6)V99`	8 digits, decimal point implied between
`PIC X(10)`	10 characters of anything
`PIC Z(6)`	digits, leading zeros replaced by spaces on output

Learn to read the file *backwards* from the PIC clause — given the record layout, count bytes and predict the contents — and you can debug any mainframe extract. `V` and `S` costing zero bytes is the insight. That's the whole trick.

2. `COMP-3` — packed decimal.

The one your lab doesn't show, and the one you'll meet in real financial data. Financial mainframe files often store numbers as `COMP-3`, where each digit takes half a byte and the last half-byte is a sign. Two digits per byte, plus one sign nibble. If you ever open what you expect to be a text file and see binary garbage where the amounts should be, that's what you're looking at. Worth reading about before you hit it.

3. The `DIVISION` structure and `SPECIAL-NAMES`.

Every COBOL program is four divisions: `IDENTIFICATION`, `ENVIRONMENT`, `DATA`, `PROCEDURE`. Your lab's `account_lookup.cbl` almost certainly has a `DATA DIVISION` where the *record layout itself* is the specification — the PIC clauses there define the exact byte offsets of `data/accounts.dat`. That's the link: the file format isn't documented somewhere, it *is* the COBOL source. If you want to extend the lab, that's the file to read first.

```
                                                    STOP RUN
RETURNING n
```

Earlier I said the 0/4/8/12 convention is the most transferable thing in the lab. The mechanism

```
                                                    STOP RUN RETURNING
<n>     return
```

setting a shared special register. Same idea, different machinery.

What to actually do

Read `/opt/balerica-bank/cobol/` rather than study COBOL generally. The source is on your instance. `account_lookup.cbl` and `payment_post.cbl` are probably under 200 lines each. Open them, find the `DATA DIVISION`, and match the PIC clauses against the actual bytes in `data/accounts.dat` and `data/payments.dat`. That exercise — source layout to real bytes, by hand — is worth more than any tutorial, and it's exactly what the runbook gestures at in Step 3 without making you do it.

If you try:

If you try:

```
 sed -n '/DATA DIVISION/,/PROCEDURE DIVISION/p' /opt/balerica-bank/cobol/*.cbl
```

…you'll see the record definitions side by side. Then `od -c data/payments.dat` shows you the bytes the PIC clauses describe. Line them up and the fixed-width idea stops being abstract.

Then, if you want the language itself, GnuCOBOL is installed on the instance (the installer used it). If there's a `.cbl` source, `cobc -x -o myprog myprog.cbl` compiles it — that's the one real COBOL command the lab contains, just hidden inside `install.sh`. Try compiling something trivial of your own; writing a `HELLO WORLD` with four divisions and a `DISPLAY` statement is maybe fifteen minutes and it'll make every COBOL file you ever open legible.

One caveat on my own answer: I'm inferring the internal structure of those `.cbl` files from the runbook's description and from how COBOL programs are conventionally written. I haven't seen them — they're on your instance, not in what you pasted. The `PIC` layout, the four divisions, and the compiled-binary-as-executable point are all reliable; the specific filenames and whether `payment_post.cbl` is named exactly that are my reconstruction from the runbook's directory listing, so treat those as likely rather than confirmed.

Generated by use.ai